SathyaNishta

Financial Investigation Platform

Architecture & Technical Documentation

Agent Workflow · System Architecture · Database Design
Error Handling · Impact Model · Business Case

Team Pink Gambit

PS6 — AI for Indian Investor

Category 3: Extending the Capabilities of MarketChatGPT by ET

Hackathon sponsored by **The Economic Times**

March 29, 2026

Technical reference document for evaluators, judges, and reviewers.
Diagrams are referenced as figures in their respective sections;
insert the corresponding image files when compiling in Overleaf.

Contents

1 Executive Summary **3**

2 Problem Statement & Motivation **3**
- 2.1 The Information Gap in Indian Retail Investing . . . 3
- 2.2 Why This Matters . . . 3

3 High-Level System Architecture **3**
- 3.1 Overview . . . 3
- 3.2 Layer Breakdown . . . 4
- 3.3 High-Level Architecture Diagram . . . 4

4 Database Design **5**
- 4.1 Schema Overview . . . 5
- 4.2 Core Tables . . . 5
  - 4.2.1 Users Table . . . 5
  - 4.2.2 Investigations Table . . . 6
  - 4.2.3 Audit Trails Table . . . 6
- 4.3 Domain-Specific Tables . . . 6
- 4.4 Supabase Storage Buckets . . . 7

5 Agent Workflow Architecture **7**
- 5.1 Overview . . . 7
- 5.2 Agent Workflow Diagram . . . 8
- 5.3 Agent-by-Agent Breakdown . . . 8
  - 5.3.1 1. Supervisor Agent . . . 8
  - 5.3.2 2. Financial Agent . . . 8
  - 5.3.3 3. News Agent . . . 9
  - 5.3.4 4. Compliance Agent . . . 9
  - 5.3.5 5. Audio Agent . . . 9
  - 5.3.6 6. Graph Agent . . . 9
  - 5.3.7 7. Reflection Node . . . 10
  - 5.3.8 8. Synthesis Agent . . . 10
- 5.4 Verdict Scale . . . 11

6 Sequence Diagram **11**
- 6.1 Key Sequence Steps . . . 11

7 Document Processing Pipeline **12**
- 7.1 Upload Flow . . . 12
- 7.2 PDF Processing . . . 12
- 7.3 Audio Processing . . . 12

8 Error Handling Logic **13**
- 8.1 Agent Timeout . . . 13
- 8.2 Database Query Failure . . . 13
- 8.3 LLM API Error . . . 13
- 8.4 Contradiction Detection . . . 13
- 8.5 Audit Trail . . . 13

9 Architecture Rationale **13**
- 9.1 Separation of Concerns . . . 13
- 9.2 Scalability . . . 14
- 9.3 Data Integrity . . . 14
- 9.4 Performance . . . 14

10 Impact Model & Business Case **14**
- 10.1 Time Saved Per Investigation . . . 14

10.2 Cost Savings 14
10.3 Revenue Model 15
10.4 Revenue Projections (Conservative, Year 1) 15
10.5 Competitive Differentiation 15

11 Integration with MarketChatGPT **16**

12 Data Flow Summary **16**

13 Acknowledgements & Credits **16**

1. Executive Summary

SathyaNishta is a multi-agent AI platform built to automate deep financial investigation of Indian-listed companies. It is submitted to the Economic Times Hackathon under **PS6 — AI for Indian Investor, Category 3: Extending the Capabilities of MarketChatGPT by ET**.

India's retail investment ecosystem has grown rapidly: over **14 crore Demat accounts** are active as of 2024, yet more than 90% of retail F&O traders lose money (SEBI FY2023–24 data). The primary cause is information asymmetry — institutional investors run teams of analysts performing research that takes 8–12 hours per company, while retail investors have no equivalent tooling.

SathyaNishta closes this gap. A single query against a company name triggers a seven-node LangGraph workflow that queries five independent data sources, cross-validates all findings through a Reflection node, and produces a structured fraud-risk verdict in approximately **4 minutes** — a **99.9% reduction** in investigation time.

The platform integrates directly into the ET Markets ecosystem (MarketChatGPT), inheriting Economic Times' data moat while adding multi-agent investigation capabilities that no existing retail tool offers.

Repository: https://github.com/Pragadhishnitt/SathyaNishta/tree/main/docs
Hackathon: Economic Times PS6 · AI for Indian Investor · Category 3

2. Problem Statement & Motivation

2.1. The Information Gap in Indian Retail Investing

Existing AI tools — including generic LLM chatbots and MarketChatGPT in its current form — can answer factual questions about stocks but cannot perform structured investigative analysis across multiple data dimensions simultaneously. No publicly available retail tool:

- Runs multi-dimensional analysis across financial filings, news sentiment, compliance records, earnings call audio, and entity relationships in a single workflow.
- Assigns an evidence-backed fraud risk score with per-dimension confidence levels.
- Maintains a full audit trail for every claim made in its output.
- Provides source citations traceable to raw database rows.

2.2. Why This Matters

- **Rs. 83,000 crore** in retail F&O losses (FY2023–24, SEBI).
- **14 crore** Demat account holders, growing at 20%+ year-on-year.
- **1.4 crore** ET Markets monthly active users — a high-intent, addressable base.
- Institutional analysts charge Rs. 8,000/hour for equivalent research.

SathyaNishta makes institutional-grade investigation available to every retail investor at the cost of a streaming subscription.

3. High-Level System Architecture

3.1. Overview

The SathyaNishta platform is a multi-tier system. All requests flow from the frontend through a reverse-proxy gateway to a Python backend, with persistent data stored in Supabase (PostgreSQL + object storage) and graph data in Neo4j.

Request Path:
`Next.js Frontend → Traefik Gateway (port 8090) → FastAPI Backend (port 8000)`
`→ Supabase (PostgreSQL + Storage) ↔ Neo4j Graph DB`

3.2. Layer Breakdown

Layer	Technology	Responsibility
Presentation	Next.js	UI, query submission, result rendering, PDF export
Gateway	Traefik	Rate limiting, TLS termination, reverse proxying
Application	FastAPI	Investigation lifecycle, authentication, webhooks
Orchestration	LangGraph	Deterministic agent routing, shared state management
AI Gateway	Portkey	LLM observability, cost tracking, fallback routing
LLM Primary	Gemini 1.5 Flash	Main model for agent analysis and synthesis
LLM Fallback	Cohere Command R+	NLP fallback when primary model is unavailable
Web Search	Tavily	Real-time news and web data retrieval
Relational DB	Supabase / PostgreSQL	Structured data, user accounts, investigations
Graph DB	Neo4j	Entity relationship graph, centrality analysis
Object Storage	Supabase Storage	Financial PDFs, audio files, uploaded documents

Table 1: System layer breakdown

3.3. High-Level Architecture Diagram

Figure 1 shows the full high-level architecture diagram including all layers, data stores, LLM routing, and inter-service communication.

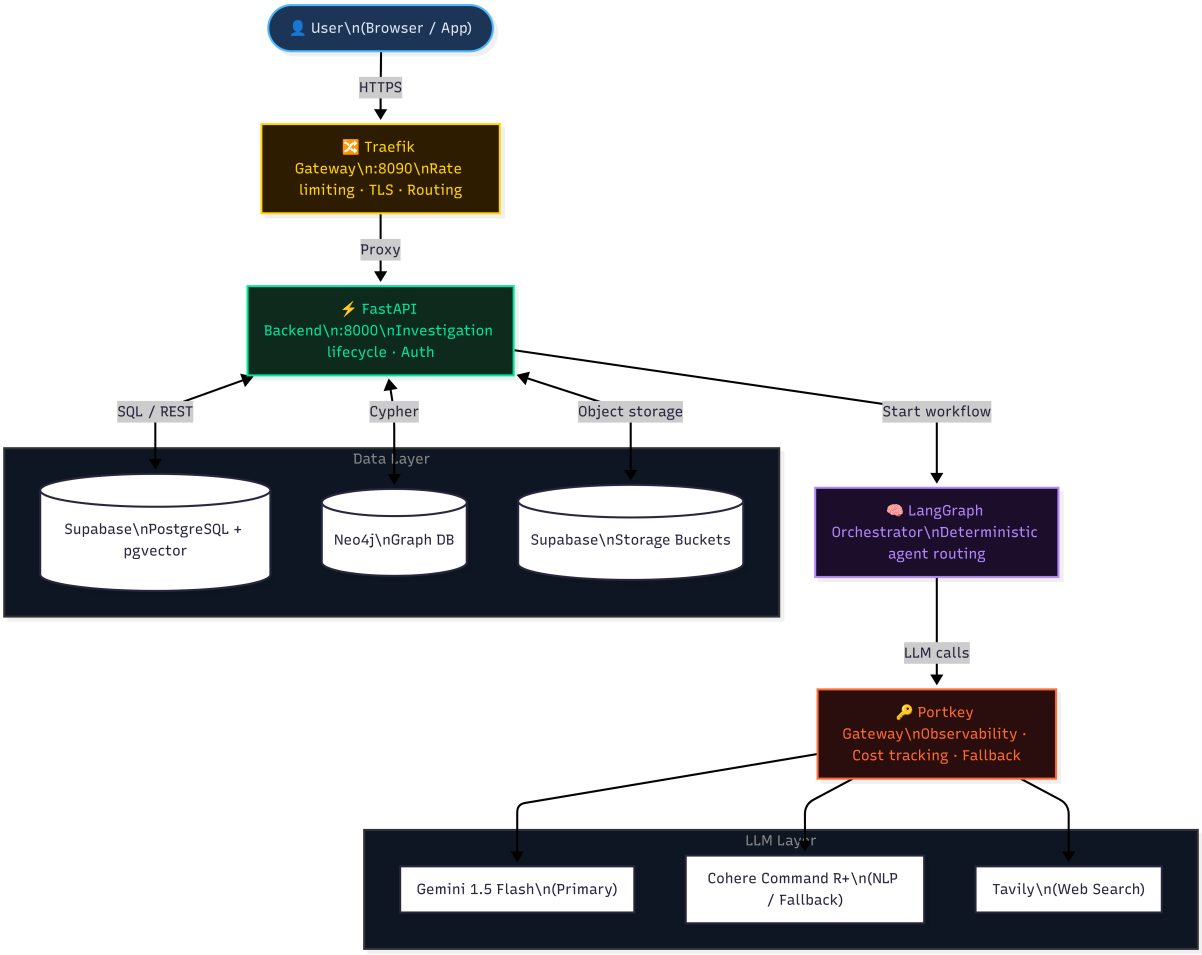

Figure 1: SathyaNishta — High-Level System Architecture

4. Database Design

4.1. Schema Overview

All structured data lives in Supabase (PostgreSQL). The schema is organised around the investigation lifecycle: users create investigations, specialist agents query domain-specific tables, and every action is recorded in audit trails.

4.2. Core Tables

4.2.1. Users Table

```sql
CREATE TABLE users (
    id            INTEGER PRIMARY KEY,
    email         VARCHAR UNIQUE NOT NULL,
    name          VARCHAR,
    password_hash VARCHAR,
    is_verified   BOOLEAN DEFAULT false,
    is_premium    BOOLEAN DEFAULT false,
    provider      VARCHAR DEFAULT 'credentials',
    created_at    TIMESTAMPTZ DEFAULT NOW(),
    last_login    TIMESTAMPTZ
);
```

Listing 1: users table DDL

Purpose: User authentication and authorisation. Supports credential-based login and OAuth providers (Google, GitHub). The `is_premium` flag gates access to the full SathyaNishta investigation mode.

4.2.2. Investigations Table

```sql
CREATE TABLE investigations (
    id                          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    query                       TEXT NOT NULL,
    status                      VARCHAR DEFAULT 'queued',
                                -- queued | running | completed | failed
    company_name                VARCHAR,
    fraud_risk_score            FLOAT,
    verdict                     VARCHAR,
                                -- critical | high | medium | low | safe
    summary                     TEXT,
    context                     JSONB DEFAULT '{}',
    created_at                  TIMESTAMPTZ DEFAULT NOW(),
    updated_at                  TIMESTAMPTZ DEFAULT NOW(),
    completed_at                TIMESTAMPTZ,
    estimated_completion_time TIMESTAMPTZ
);
```

Listing 2: investigations table DDL

Purpose: Tracks the full lifecycle of every investigation. Status transitions: `queued` → `running` → `completed` (or `failed`). The `fraud_risk_score` and `verdict` are populated by the Synthesis agent on completion.

4.2.3. Audit Trails Table

```sql
CREATE TABLE audit_trails (
    id                INTEGER PRIMARY KEY,
    investigation_id UUID REFERENCES investigations(id),
    step_type         VARCHAR,  -- document_processing | agent_start | synthesis
    agent_type        VARCHAR,  -- financial_agent | news_agent | ...
    input_payload     JSONB,
    output_payload    JSONB,
    model_metadata    JSONB,
    timestamp         TIMESTAMPTZ DEFAULT NOW()
);
```

Listing 3: audit_trails table DDL

Purpose: Records every agent action, input, and output for transparency, debugging, and regulatory compliance. Every step is logged before the next begins.

4.3. Domain-Specific Tables

The five specialist agents each query a dedicated table. Long column lists are abbreviated below; full DDL is in the repository.

Table	Key Columns	Purpose & Agent
`financial_filings`	symbol, filing_type, revenue, net_income, total_assets, content_chunk	Parsed financial statements (10-K, 10-Q). **Financial Agent.**
`news_articles`	title, content, source, published_at, sentiment_score, relevance_score	News with pre-computed sentiment $[-1, +1]$. **News Agent.**
`compliance_records`	entity, violation_type, severity, fine_amount, status	Regulatory violations and fines. **Compliance Agent.**
`audio_transcripts`	content, speaker, company, sentiment_score, source_file_key	Earnings call transcriptions. **Audio Agent.**
`chat_threads`	user_id, mode, investigation_id	Per-user persistent chat sessions.
`chat_messages`	thread_id, role, content, extra_data	Individual messages within chat threads.

Table 2: Domain-specific database tables and their consuming agents

4.4. Supabase Storage Buckets

Bucket	File Types / Limit	Path Structure & Notes
`financial_docs`	PDF, max 50 MB	`{ticker}/{FY}/{period}/{doc}.pdf` — Financial Agent
`audio_recordings`	MP3, WAV, M4A, max 200 MB	`{ticker}/{FY}/{period}/earnings_{date}.{ext}` — Audio Agent
`temp_uploads`	Any, max 100 MB	`{uuid}/{filename}` — Staging; router moves to permanent bucket
`news_uploads`	HTML, TXT, JSON, max 50 MB	`{source}/{date}/{cat}/{file}` — News Agent

Table 3: Supabase object storage buckets

5. Agent Workflow Architecture

5.1. Overview

The investigation engine is a seven-node **LangGraph** state machine. Routing is entirely deterministic: the Supervisor node reads a static `AGENT_SEQUENCE` dictionary and sets `next_agent` without any LLM call. This eliminates non-determinism in routing — a critical requirement for a financial tool where reproducibility and auditability are paramount.

Agent Execution Order:
Supervisor → Financial → News → Compliance → Audio → Graph → Reflection → Synthesis → Verdict

5.2. Agent Workflow Diagram

Figure 2 shows the multi-agent workflow: five specialist agents running sequentially under Supervisor control, converging into the Reflection node, and terminating with Synthesis and Verdict.

Figure 2: SathyaNishta — Multi-Agent Workflow (LangGraph)

5.3. Agent-by-Agent Breakdown

5.3.1. 1. Supervisor Agent

- **Data source:** `investigations` table (routing context only)
- **LLM call:** None — purely deterministic Python logic
- **Output key:** `next_agent`

```
def supervisor_node(state):
    current_agent = state.get("current_agent", "financial")
    next_agent = AGENT_SEQUENCE[current_agent]
    return {"next_agent": next_agent}
```

Listing 4: Supervisor node

The Supervisor enforces strict sequential execution. There is no probabilistic routing and no shared mutable state between specialist agents. Each agent receives the company name and writes exactly one typed output key back to the shared state object.

5.3.2. 2. Financial Agent

- **Data source:** `financial_filings`
- **Analysis:** Balance sheets, income statements, financial ratios, P&L trends, debt anomalies
- **Output key:** `financial_findings`

```
def financial_node(state):
    company  = state["company_name"]
    filings  = query_financial_filings(company)
    analysis = llm_analyze(filings)
    return {"financial_findings": analysis}
```

Listing 5: Financial Agent node

```
SELECT * FROM financial_filings
WHERE  LOWER(company_name) = LOWER(:company)
ORDER  BY filing_date DESC;
```

Listing 6: Financial Agent SQL query

5.3.3. 3. News Agent

- **Data source:** `news_articles`
- **Analysis:** Sentiment scoring $[-1, +1]$, relevance scoring $[0, 1]$, reputational risk flagging
- **Output key:** `news_findings`

```
SELECT * FROM news_articles
WHERE  content ILIKE '%' || :company || '%'
ORDER  BY published_at DESC
LIMIT  10;
```

Listing 7: News Agent SQL query

5.3.4. 4. Compliance Agent

- **Data source:** `compliance_records`
- **Analysis:** Violation classification, fine quantification, severity-weighted risk score
- **Output key:** `compliance_findings`

```
SELECT * FROM compliance_records
WHERE  LOWER(entity) = LOWER(:company)
ORDER  BY date DESC;
```

Listing 8: Compliance Agent SQL query

5.3.5. 5. Audio Agent

- **Data source:** `audio_transcripts`
- **Analysis:** Hedging language detection, management tone, evasive pattern identification, guidance extraction
- **Output key:** `audio_findings`

```
SELECT * FROM audio_transcripts
WHERE  LOWER(company) = LOWER(:company)
ORDER  BY date DESC;
```

Listing 9: Audio Agent SQL query

5.3.6. 6. Graph Agent

- **Data source:** Neo4j graph database (Cypher queries)
- **Analysis:** Entity relationship graph construction, network centrality computation, hidden connection mapping
- **Output key:** `graph_findings`

```
def graph_node(state):
    company       = state["company_name"]
    relationships = build_entity_graph(company)
    centrality    = calculate_centrality(relationships)
    return {"graph_findings": centrality}
```

Listing 10: Graph Agent node

5.3.7. 7. Reflection Node

- **Input:** All five specialist agent output keys
- **Processing:** Cross-dimensional consistency checking, per-finding confidence scoring [0.0, 1.0], contradiction flagging
- **Output key:** `validated_findings`

```
def reflection_node(state):
    all_findings = {
        "financial":  state["financial_findings"],
        "news":       state["news_findings"],
        "compliance": state["compliance_findings"],
        "audio":      state["audio_findings"],
        "graph":      state["graph_findings"],
    }
    validated = cross_validate_findings(all_findings)
    return {"validated_findings": validated}
```

Listing 11: Reflection Node

The Reflection node is the system's quality gate. When findings contradict each other (e.g. strong financial metrics alongside critical compliance violations), it raises the minimum confidence threshold required before Synthesis proceeds and surfaces contradictions explicitly in the final report.

5.3.8. 8. Synthesis Agent

- **Input:** `validated_findings` from the Reflection node
- **Processing:** LLM holistic analysis, fraud risk score computation, verdict determination
- **Output:** Structured report + `fraud_risk_score` + `verdict`

```
def synthesis_node(state):
    all_findings = state["validated_findings"]
    final_report = llm_synthesize(all_findings)
    risk_score   = calculate_overall_risk(all_findings)
    return {
        "final_report": final_report,
        "risk_score":   risk_score,
        "verdict":      determine_verdict(risk_score),
    }
```

Listing 12: Synthesis Agent node

5.4. Verdict Scale

Verdict	Score Range	Meaning
`critical`	0.85 – 1.00	Multiple severe red flags across dimensions; immediate concern
`high`	0.65 – 0.84	Significant risk signals in two or more dimensions
`medium`	0.40 – 0.64	Moderate concerns; warrants further monitoring
`low`	0.20 – 0.39	Minor issues; generally within acceptable bounds
`safe`	0.00 – 0.19	No meaningful risk signals across all dimensions

Table 4: Fraud risk verdict scale

6. Sequence Diagram

Figure 3 shows the complete sequence diagram across all system participants: User, Next.js Frontend, Traefik Gateway, FastAPI Backend, LangGraph, Agents (×5), Supabase DB, and Portkey → Gemini.

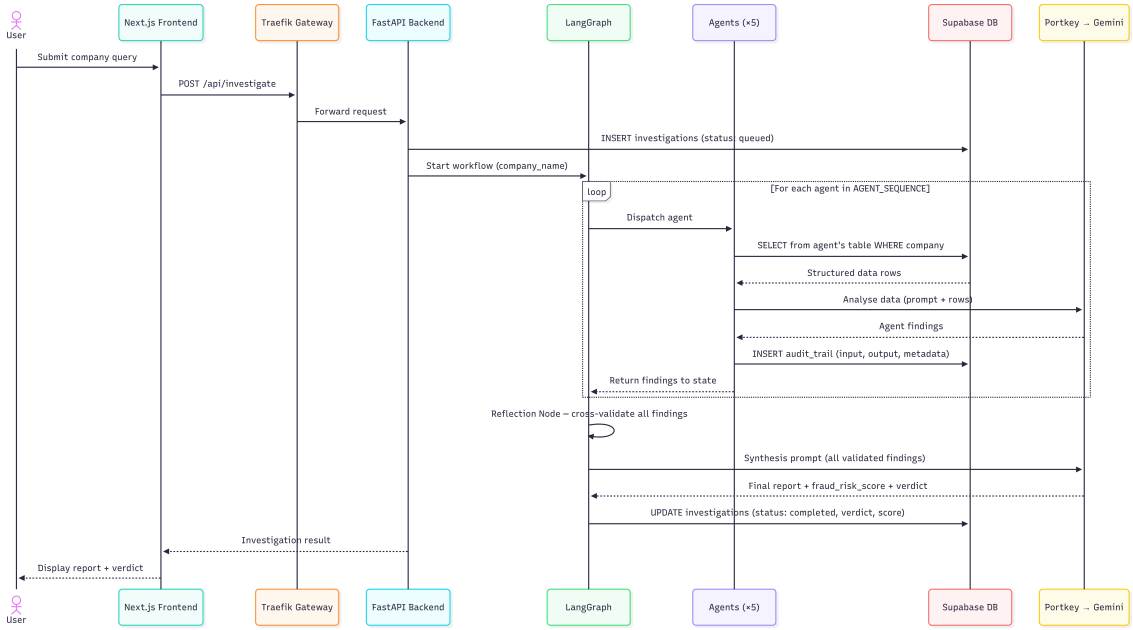

Figure 3: SathyaNishta — End-to-End Sequence Diagram

6.1. Key Sequence Steps

1. User submits company query via the Next.js frontend.
2. Frontend sends `POST /api/investigate` through the Traefik gateway.
3. FastAPI inserts an Investigation record with `status: queued` into Supabase.
4. FastAPI triggers the LangGraph workflow with `company_name` in state.
5. **Loop — for each agent in AGENT_SEQUENCE:**
   - Supervisor dispatches the next agent.
   - Agent issues `SELECT` against its assigned DB table.

- Supabase returns structured data rows.
- Agent sends data + prompt to Portkey → Gemini 1.5 Flash.
- LLM returns agent findings.
- Agent inserts an audit trail record (input, output, metadata).
- Agent writes findings back to LangGraph shared state.

6. Reflection node cross-validates all five findings.
7. Synthesis agent sends the full validated context to the LLM.
8. LLM returns final report + `fraud_risk_score` + `verdict`.
9. FastAPI updates Investigation record to `status: completed`.
10. FastAPI delivers result to the frontend via SSE/polling.
11. Frontend displays the full report and verdict to the user.

7. Document Processing Pipeline

7.1. Upload Flow

1. User uploads file → `temp_uploads` bucket.
2. Supabase Storage event triggers an Edge Function.
3. Edge Function calls a FastAPI backend webhook.
4. FastAPI processes the document as an async background task.
5. Text is extracted and parsed; structured data is inserted into the relevant DB table.
6. A PostgreSQL trigger fires and auto-creates an Investigation record.
7. File is moved from `temp_uploads` to the appropriate permanent bucket.

7.2. PDF Processing

```
# 1. Download from Supabase storage
file_content = download_from_bucket('financial_docs', file_path)

# 2. Extract text
text = PyPDF2.extract_text(file_content)

# 3. Parse structured financial data
financial_data = parse_financial_statements(text)

# 4. Store in database
insert_financial_filings(financial_data, file_path)
```

Listing 13: Financial PDF processing pipeline

7.3. Audio Processing

```
# 1. Download audio file
audio_content = download_from_bucket('audio_recordings', file_path)

# 2. Transcribe via speech-to-text
transcript = speech_to_text(audio_content)

# 3. Analyse sentiment and extract insights
sentiment = analyze_sentiment(transcript)

# 4. Store transcription and metadata
insert_audio_transcript(transcript, sentiment, file_path)
```

Listing 14: Earnings call audio processing pipeline

8. Error Handling Logic

8.1. Agent Timeout

Each specialist agent runs with a configurable timeout. On expiry the agent returns a partial result with `confidence = 0` and tags the finding as `UNVERIFIED`. The Reflection node records the gap explicitly; the Synthesis agent downweights that dimension when computing the final risk score. The investigation is not aborted — it proceeds with reduced confidence.

8.2. Database Query Failure

If a query returns zero rows (e.g. an unknown ticker symbol or a company with no compliance records on file), the agent emits an empty findings object rather than raising an exception. The Synthesis agent treats a missing dimension as neutral and notes the data unavailability explicitly in the final report.

8.3. LLM API Error

Every LLM call is wrapped in a `try/except` block with exponential back-off. Portkey Gateway provides an automatic fallback from Gemini 1.5 Flash to Cohere Command R+ before the retry limit is reached.

- Retry 1: after 1 second
- Retry 2: after 2 seconds
- Retry 3: after 4 seconds

After three consecutive failures the Investigation record is set to `status: failed` with a descriptive `error_message` field.

8.4. Contradiction Detection

When the Reflection node identifies conflicting signals across dimensions (e.g. strong financial metrics alongside critical compliance violations), it raises the minimum confidence threshold required before the Synthesis agent proceeds. Low-confidence contradictions are surfaced in the final report under a dedicated *Conflicting Signals* section rather than being silently resolved.

8.5. Audit Trail

Every agent action — input payload, output payload, model metadata, token counts, and UTC timestamp — is appended to `audit_trails` before the workflow moves to the next node. This provides full debugging transparency, supports post-hoc analysis, and satisfies regulatory compliance requirements. Every claim in the final report is traceable to a specific agent output and the raw database row it was derived from.

9. Architecture Rationale

9.1. Separation of Concerns

The architecture maintains strict separation across four layers:

- **Storage layer:** Original files and large binary objects (Supabase Storage, Neo4j).
- **Database layer:** Structured, queryable, indexed data (PostgreSQL).
- **Agent layer:** Single-responsibility analysis nodes, each scoped to one data source.
- **Trigger layer:** Automation and real-time processing (Edge Functions, PostgreSQL triggers).

Each specialist agent is scoped to one data source and one output key, preventing cross-contamination of findings before the Reflection node validates them, and making agents independently testable.

9.2. Scalability

- Async processing (FastAPI background tasks) prevents request blocking on long-running investigations.
- PostgreSQL triggers enable event-driven automation without polling.
- New agents (e.g. Insider Trading Agent, Regulatory Filing Agent) can be added to `AGENT_SEQUENCE` without modifying existing agent code.

9.3. Data Integrity

- Audit trails record every step before the next step begins.
- Foreign key constraints maintain relational integrity.
- `JSONB` columns provide schema-flexible metadata storage.
- `pgvector` extension supports future semantic similarity search over financial document embeddings.

9.4. Performance

- Indexed queries on `company_name`, `entity`, and `symbol` enable sub-second agent lookups.
- Portkey Gateway caches LLM responses for identical prompts within a session.
- Connection pooling via Supabase's built-in pooler prevents database connection exhaustion under concurrent investigations.

10. Impact Model & Business Case

10.1. Time Saved Per Investigation

Task	Manual	Platform	Method
Read annual report + financials	3–4 hrs	∼45 s	Financial Agent; pre-parsed DB
News sentiment sweep (90 days)	45–90 min	∼30 s	News Agent; `news_articles`
Compliance & regulatory check	1–2 hrs	∼45 s	Compliance Agent; violation DB
Earnings call transcript review	60–90 min	∼30 s	Audio Agent; pre-transcribed DB
Entity relationship mapping	2–3 hrs	∼10 s	Graph Agent; pre-built Neo4j graph
Synthesis + final write-up	30–60 min	∼30 s	Synthesis Agent + LLM
Total per investigation	**8–12 hrs**	**∼4 min**	**99.9% reduction**

Table 5: Per-investigation time savings

10.2. Cost Savings

Retail investor (3 investigations/week):

$$\text{Time saved} = 3 \times 10\,\text{hrs} = 30\,\text{hrs/week}$$
$$\text{Opportunity cost} = 30 \times \text{Rs.}750 = \text{Rs.}22{,}500\,/\text{week}$$
$$\text{Annual saving} \approx \textbf{Rs. 11.7 lakh/year per investor}$$

Institutional analyst (10 investigations/week):

$$\text{Time saved} = 10 \times 10\,\text{hrs} = 100\,\text{hrs/week}$$
$$\text{Opportunity cost} = 100 \times \text{Rs.8,000} = \text{Rs.8,00,000}\,/\text{week}$$
$$\text{Annual saving} \approx \textbf{Rs. 4.1 crore/year per analyst seat}$$

10.3. Revenue Model

Tier	Price	Target	Inclusions
Freemium	Rs. 0 / month	Retail investors	3 investigations/month, basic summary, news overview
Pro Investor	Rs. 299 / month	Active traders	Unlimited investigations, all 5 agents, compliance deep-dive, PDF export
Enterprise	Rs. 2L+ / yr / seat	Brokers, PMS, MFs	API access, white-label, custom ingestion, SLA support

Table 6: Proposed revenue tiers

10.4. Revenue Projections (Conservative, Year 1)

$$\text{ET Markets MAU} = 1{,}40{,}00{,}000$$
$$\text{Premium conversion @ 1\%} = 1{,}40{,}000 \text{ subscribers}$$
$$\text{MRR} = 1{,}40{,}000 \times \text{Rs.299} = \text{Rs.4.19 crore}$$
$$\text{ARR @ 1\% conversion} \approx \textbf{Rs. 50 crore}$$
$$\text{ARR @ 10\% conversion} \approx \textbf{Rs. 500 crore}$$
$$\text{B2B enterprise (30 seats)} = 30 \times \text{Rs.2L} = \text{Rs.6 crore ARR}$$
$$\textbf{Total Year 1 (conservative)} \approx \textbf{Rs. 56 crore combined ARR}$$

10.5. Competitive Differentiation

Capability	ChatGPT	MarketChatGPT	SathyaNishta
Real-time Indian stock data	×	✓	✓
Multi-agent investigation (5 agents)	×	×	✓
Compliance + regulatory check	×	×	✓
Earnings call audio analysis	×	×	✓
Entity graph relationships	×	×	✓
Fraud risk score + verdict	×	×	✓
Full audit trail transparency	×	×	✓
Source citations on every claim	×	Partial	✓

Table 7: Competitive capability matrix

11. Integration with MarketChatGPT

SathyaNishta is designed as a native extension layer on MarketChatGPT by Economic Times, not as a standalone competitor. The integration rationale is as follows:

- **Data moat:** ET Markets already aggregates real-time Indian financial data, news, and market feeds. SathyaNishta consumes this data as its primary input without replicating the underlying infrastructure.
- **User base:** ET Markets' 1.4 crore MAU are the exact target persona — high-intent retail investors seeking deeper analysis.
- **Surface:** SathyaNishta adds a "Deep Investigation" mode to MarketChatGPT. The existing conversational Q&A mode remains unchanged; the seven-agent workflow activates only when a user requests a full company investigation.
- **Monetisation:** The Pro tier (Rs. 299/month) is a natural upsell path within the existing ET subscription ecosystem.

12. Data Flow Summary

Stage	Component	Action
Upload	`temp_uploads` → Edge Function	File classified and routed to the appropriate storage bucket
Ingestion	FastAPI background task	Text extracted; structured data inserted into the relevant DB table
Trigger	PostgreSQL trigger / webhook	Investigation record created automatically on new data
Analysis	Specialist agents (×5)	Each agent queries its own DB table and produces typed findings
Validation	Reflection node	Findings cross-validated; confidence scores assigned
Synthesis	Synthesis Agent → PostgreSQL	Final report and `fraud_risk_score` persisted to investigations
Delivery	FastAPI SSE / polling	Frontend receives status update and renders the report

Table 8: End-to-end data flow summary

13. Acknowledgements & Credits

- **The Economic Times** — for organising and sponsoring this hackathon and for providing MarketChatGPT as an extensible foundation for PS6 submissions.
- **ET Markets** — for the real-time Indian financial data infrastructure that powers SathyaNishta's data layer.
- **LangGraph / LangChain** — for the agent orchestration framework.
- **Supabase** — for managed PostgreSQL and object storage.
- **Neo4j** — for the graph database powering entity relationship analysis.
- **Portkey** — for LLM gateway, observability, and fallback routing.

- **Google Gemini / Cohere** — for the language model capabilities underpinning agent analysis and synthesis.

Team Pink Gambit · PS6: AI for Indian Investor · Category 3: Extending the Capabilities of MarketChatGPT by ET
Economic Times Hackathon · March 2026

Repository: https://github.com/Pragadhishnitt/SathyaNishta/tree/main/docs